Guía de pruebas en Postman

TPI Logística de última milla · Backend de Aplicaciones 2026 · Comisión 3K4

Esta guía explica cómo levantar el sistema completo en local y probar todos los endpoints desde Postman con la colección del repositorio (`postman/TPI-Logistica.postman_collection.json`). Cubre el flujo principal de punta a punta, los escenarios alternativos (cancelación, entrega fallida, reprogramación), las integraciones externas (Google Routes API y Gmail), la tolerancia a caídas y las pruebas de seguridad.

Todas las requests van contra el **api-gateway** (`http://localhost:8080`). Nunca se llama directo a los microservicios: así se prueba también el ruteo y la validación del token.

1. Requisitos

Herramienta	Versión	Para qué
Postman (app de escritorio)	cualquiera reciente	Ejecutar la colección. La versión web necesita el Postman Agent para llegar a localhost.
JDK	21 o superior	Compilar y correr los servicios.
Maven	3.9 o superior	`mvn spring-boot:run` y `mvn install`.
PostgreSQL	17	Una base por servicio: `pedidos_db` y `despacho_db`.
Keycloak	26	Login y emisión de tokens. Se descarga como .zip desde keycloak.org y se descomprime.

Variables de entorno opcionales. Sin ellas el sistema funciona igual, con las limitaciones indicadas:

Variable	Si no está
`GOOGLE_MAPS_API_KEY`	Las distancias se estiman en línea recta y las rutas salen con `estimada = true`.
`GMAIL_APP_PASSWORD`	No se envían mails de cambio de estado (solo queda una línea en el log).

Se cargan una sola vez en PowerShell y quedan permanentes para tu usuario de Windows:

```
setx GOOGLE_MAPS_API_KEY "tu-api-key"
setx GMAIL_APP_PASSWORD "xxxxxxxxxxxxxxxx"
```

> **Importante.** después de un `setx` hay que cerrar y volver a abrir la terminal (o IntelliJ) para que la variable se vea. Las claves nunca se escriben en archivos del repositorio.

2. Levantar el sistema

El orden importa: cada servicio necesita que lo anterior ya esté corriendo. Cada comando va en su propia terminal, que queda abierta mientras se prueba.

Paso 1 · Bases de datos (solo la primera vez)

Con PostgreSQL corriendo, desde la carpeta del repositorio:

```
psql -U postgres -f infra/init-db.sql
```

Crea las dos bases con sus usuarios. Las tablas las crea cada servicio al arrancar.

Paso 2 · Keycloak

Copiar `infra/realm-logistica.json` a la carpeta `data/import` de Keycloak (crearla si no existe) y arrancarlo **desde la carpeta de Keycloak**:

```
cd C:\ruta\a\keycloak-26.x.x
.\bin\kc.bat start-dev --http-port=8180 --import-realm
```

Está listo cuando aparece `Listening on: http://0.0.0.0:8180`. El realm `logistica` trae los roles, los clients y los usuarios de prueba.

Paso 3 · Librería común (la primera vez y cada vez que cambie)

```
cd logistica-commons
mvn clean install
```

Paso 4 · Microservicios, en este orden

```
cd pedidos-service   ;  mvn spring-boot:run     # puerto 8081
cd despacho-service  ;  mvn spring-boot:run     # puerto 8082
cd api-gateway       ;  mvn spring-boot:run     # puerto 8080
```

Cada uno está listo cuando muestra `Started ...Application`. También se pueden correr desde IntelliJ con la clase `*Application` de cada proyecto.

Componente	Puerto	Comprobación rápida en el navegador
Keycloak	8180	`http://localhost:8180/realms/logistica` devuelve un JSON
pedidos-service	8081	`http://localhost:8081/swagger-ui.html`
despacho-service	8082	`http://localhost:8082/swagger-ui.html`
api-gateway	8080	`http://localhost:8080/actuator/health` devuelve UP

> **Datos iniciales.** despacho-service carga solo, al arrancar, los datos de prueba: 2 centros (1 = Córdoba, 2 = Neuquén), 4 vehículos y 3 repartidores. No hace falta crearlos a mano.

3. Importar y entender la colección

En Postman: botón **Import** (arriba a la izquierda) y arrastrar el archivo `postman/TPI-Logistica.postman_collection.json`. Aparece la colección **TPI Logística** con una carpeta por rol:

Carpeta	Usuario	Qué contiene
1. Cliente	`cliente1`	Perfil del comercio, alta de pedidos, consulta, historial y cancelación.
2. Despachante	`despachante1`	Planificación, rutas, reprogramación, distancias, notificaciones y lectura de datos maestros.
3. Repartidor	`repartidor1`	Mi ruta del día, inicio de ruta y resultado de cada parada.
4. Admin	`admin1`	Alta, modificación y baja de centros, vehículos y repartidores; desactivar clientes.

Login automático

No hace falta pedir tokens a mano. Cada carpeta tiene un script (pestaña **Scripts** → **Pre-request** de la carpeta) que antes de cada request inicia sesión en Keycloak con su usuario (la contraseña es igual al usuario), guarda el token y lo reutiliza hasta que vence. Para probar con otro usuario, se cambia la primera línea de ese script:

```
const usuario = 'repartidor1';   // por ejemplo, 'repartidor2'
```

Variables de la colección

Se ven y editan en la colección → pestaña **Variables**. Las requests que crean algo guardan el id automáticamente, así las siguientes ya lo usan:

Variable	Valor inicial	Quién la completa
`gateway`	`http://localhost:8080`	Fija.
`keycloak`	`http://localhost:8180`	Fija.
`centroId`	`1`	Fija (centro Córdoba). Cambiarla para probar otro centro.
`pedidoId`	vacía	Cliente → Crear pedido.
`rutaId`	vacía	Despachante → Planificar día, y Repartidor → Mi ruta de hoy.
`paradaId`	vacía	Repartidor → Mi ruta de hoy / Iniciar ruta (primera parada PENDIENTE).
`hoy`	calculada	Se recalcula en cada request con la fecha local (AAAA-MM-DD).

4. Flujo principal: del alta a la entrega

Ejecutar en este orden con **Send**. Es el recorrido completo de un pedido; al final el historial muestra cada cambio con fecha, motivo y autor.

#	Carpeta → Request	Resultado esperado
1	Cliente → Registrar mi perfil	**201** con el perfil. Si ya se registró antes da **409** (perfil o CUIT duplicado): se sigue igual.
2	Cliente → Crear pedido	**201**, `"estado": "RECIBIDO"`. El id queda en `pedidoId`. La ventana horaria es hoy de 09:00 a 18:00.
3	Despachante → Planificar día	**200** con `"estado": "CONFIRMADA"` y una ruta en `rutas` que contiene el pedido. Mirar el campo `repartidor` (ver nota abajo).
4	Cliente → Ver pedido	`"estado": "ASIGNADO"`.
5	Repartidor → Mi ruta de hoy	**200** con la ruta en estado `CONFIRMADA` y sus paradas. Guarda `rutaId` y `paradaId`.
6	Repartidor → Iniciar ruta	**200**, ruta `EN_CURSO`. El pedido pasa a `EN_CAMINO` en segundos.
7	Repartidor → Parada entregada	**200**, parada `ENTREGADA`. Con la última parada la ruta pasa a `FINALIZADA`.
8	Cliente → Historial del pedido	RECIBIDO → ASIGNADO → EN_CAMINO → ENTREGADO, cada uno con fecha, motivo y autor.

> **Qué repartidor recibe la ruta.** despacho-service tiene dos repartidores en Córdoba: **Juan Pérez** (`repartidor1`, turno 08 a 16) y **María Gómez** (`repartidor2`, turno 12 a 20). El planificador elige el que da la ruta más conveniente. Si en el paso 3 la ruta quedó para María Gómez, cambiar el usuario de la carpeta Repartidor a `repartidor2` (sección 3) antes del paso 5; si no, **Mi ruta de hoy** responde 404.

Si se configuró el mail (sección 6.2), en el paso 8 el email del perfil del cliente ya recibió un mail por cada cambio de estado.

5. Escenarios alternativos

5.1 Cancelación

#	Request	Resultado esperado
1	Cliente → Crear pedido	**201**, RECIBIDO (pisa `pedidoId` con el nuevo).
2	Cliente → Cancelar pedido	**200**, `CANCELADO`. Si estaba ASIGNADO, primero se quita de la ruta.
3	Cliente → Cancelar pedido (otra vez)	**422**: CANCELADO es un estado final.

Para ver el rechazo por pedido en la calle: cancelar un pedido que ya está EN_CAMINO (después del paso 6 del flujo principal). Responde **422**: no se puede cancelar algo que ya salió.

5.2 Entrega fallida

Repetir el flujo principal con otro pedido, pero en el paso 7 usar **Repartidor** → **Parada fallida** (lleva el motivo obligatorio `"Destinatario ausente"`):

- Primer intento fallido: el pedido pasa por `FALLIDO` y vuelve a `RECIBIDO`, listo para planificarse otro día.
- Segundo intento fallido: el pedido termina en `DEVUELTO` (máximo 2 intentos).
- Sin motivo, `{ "resultado": "FALLIDA" }` responde **400**.

5.3 Pedido no asignable y reprogramación

En Cliente → Crear pedido, cambiar las coordenadas del destinatario por unas fuera del radio de 25 km del centro Córdoba, por ejemplo Buenos Aires:

```
"latitud": -34.6037,
"longitud": -58.3816
```

#	Request	Resultado esperado
1	Despachante → Planificar día	El pedido aparece en `noAsignables` con el motivo (fuera de cobertura).
2	Cliente → Ver pedido	`NO_ASIGNABLE`.
3	Despachante → Reprogramar pedido	**200**, vuelve a `RECIBIDO` para la próxima planificación.

5.4 Planificación repetida o en el pasado

- Planificar con una fecha anterior a hoy (editar `"fecha"` en el body): **409**.
- Si una planificación queda `PENDIENTE_CONFIRMACION` (respuesta **202**, pedidos-service no confirmó), se reintenta con **Despachante → Reintentar confirmación**.

6. Integraciones externas

6.1 Google Routes API (distancias reales)

Despachante → Distancia entre dos puntos consulta la distancia y el tiempo por calle entre dos coordenadas. El campo `fuente` indica de dónde salió el dato:

fuente	Significado
`GOOGLE`	Consultado a Google en este momento. La API key funciona.
`CACHE`	Ya se había consultado y se reutiliza (vigencia de 30 días).
`CACHE_VENCIDA`	Google no respondió y se usó un dato viejo de la caché.
`ESTIMADA`	Sin key o sin Google: línea recta × 1,3 a 30 km/h.
`MISMO_PUNTO`	Origen y destino iguales.

En la respuesta de **Planificar día**, `fuentesDistancia` cuenta cuántos pares salieron de cada fuente y cada ruta tiene `estimada`: `false` si todas sus distancias son reales.

> **Diagnóstico.** si siempre sale `ESTIMADA` teniendo la key cargada, revisar el log de despacho-service: un 403 de Google significa que Routes API no está habilitada en el proyecto de Google Cloud de esa key.

6.2 Mails de cambio de estado (Gmail)

pedidos-service manda un mail al **email del perfil del cliente** en el alta del pedido y en cada cambio de estado. Sale desde `ultimamilla.tpbackend@gmail.com` después de que el cambio se guarda, y si Gmail falla el pedido no se ve afectado.

- Para recibirlos, registrar el perfil (Cliente → Registrar mi perfil, o Actualizar mi perfil) con un email real al que tengas acceso.
- Correr el flujo principal: llegan mails con asunto `Pedido #N: recibido`, `asignado a una ruta`, `en camino` y `entregado`.
- En el log de pedidos-service aparece `Mail enviado a ...` por cada uno. Si dice `No se pudo enviar`, la contraseña de aplicación es incorrecta.

7. Tolerancia a caídas

Prueba que el repartidor puede seguir trabajando aunque pedidos-service esté caído: los avisos quedan en una cola (outbox) y se reenvían cada 15 segundos.

#	Acción	Resultado esperado
1	Flujo principal hasta el paso 6 (ruta iniciada).	Pedido EN_CAMINO.
2	Detener pedidos-service (Ctrl+C en su terminal).	-
3	Repartidor → Parada entregada.	**200** igual: despacho registra la entrega aunque pedidos esté caído.
4	Despachante → Notificaciones con error, cambiando el parámetro `estado` a `PENDIENTE`.	Aparece el aviso de resultado de entrega pendiente.
5	Volver a levantar pedidos-service.	En menos de 15 s el aviso queda `ENVIADA` y el pedido pasa a ENTREGADO.

Si un aviso agota los reintentos queda en `ERROR` y se puede volver a encolar con `POST /api/despacho/notificaciones/{id}/reintento`.

8. Pruebas de seguridad

Cada caso se arma con una request existente, cambiando solo lo indicado. Se espera el rechazo, no un error del servidor.

Caso	Cómo	Esperado
Sin token	En cualquier request: pestaña Authorization → `No Auth`.	**401**
Cliente planificando	Duplicar Despachante → Planificar día dentro de la carpeta Cliente.	**403**
Repartidor viendo pedidos	Duplicar Cliente → Mis pedidos dentro de la carpeta Repartidor.	**403**
Despachante creando centros	Duplicar Admin → Crear centro dentro de la carpeta Despachante.	**403**
Endpoint interno por el gateway	Cualquier request con la URL `{{gateway}}/internal/pedidos/pendientes?centroId=1`.	**403** (el gateway bloquea /internal)
Pedido ajeno	Con el usuario de la carpeta Cliente cambiado a otro cliente, Ver pedido de `cliente1`.	**403**
Ruta ajena	Con `repartidor2`, Iniciar ruta usando el `rutaId` de `repartidor1`.	**403**
Datos inválidos	Crear pedido con `"pesoKg": -5` o sin `"centroId"`.	**400** con el detalle del campo

9. Problemas comunes

Síntoma	Causa y solución
Todas las requests fallan con `ECONNREFUSED` a 8080	El gateway no está corriendo. Arrancarlo (sección 2, paso 4).
La consola de Postman muestra `Login fallido para ...`.	Keycloak no está corriendo o no importó el realm. Revisar que `realm-logistica.json` esté en `data/import` y que se haya arrancado con `--import-realm`.
401 en todas las requests	Token vencido o emitido por otro Keycloak. En la colección → Variables, borrar los valores de las variables `token_*` y reintentar.
403 inesperado	La request está en una carpeta cuyo rol no tiene permiso. Ver la tabla de la sección 3.
503	El otro microservicio no responde (por ejemplo, crear un pedido con despacho-service caído). Levantarlo y reintentar.
Mi ruta de hoy da **404**	La ruta quedó para otro repartidor, o todavía no se planificó. Ver la nota de la sección 4.
Iniciar ruta da **409**	La ruta no está CONFIRMADA (quedó en BORRADOR) o ya se inició. Reintentar la confirmación.
URL con `{{pedidoId}}` vacío o 404	Falta ejecutar antes la request que crea ese recurso (por ejemplo, Crear pedido).
Planificar no asigna nada (`SIN_RUTAS`)	No hay pedidos RECIBIDO para ese centro y fecha, o la ventana horaria del pedido no es de hoy.